

State

1. Problema

En el sistema de agendamiento de citas médicas, se detectó que la lógica para manejar el **estado de una cita** (por ejemplo: `Disponible`, `Reservada`, `Cancelada`, `Completada`, etc.) estaba dispersa en condicionales (`if` / `switch`) por varias partes del código. Esto hacía difícil **agregar nuevos estados** o modificar comportamientos específicos, además de provocar **alta complejidad y bajo mantenimiento**.

2. Solución

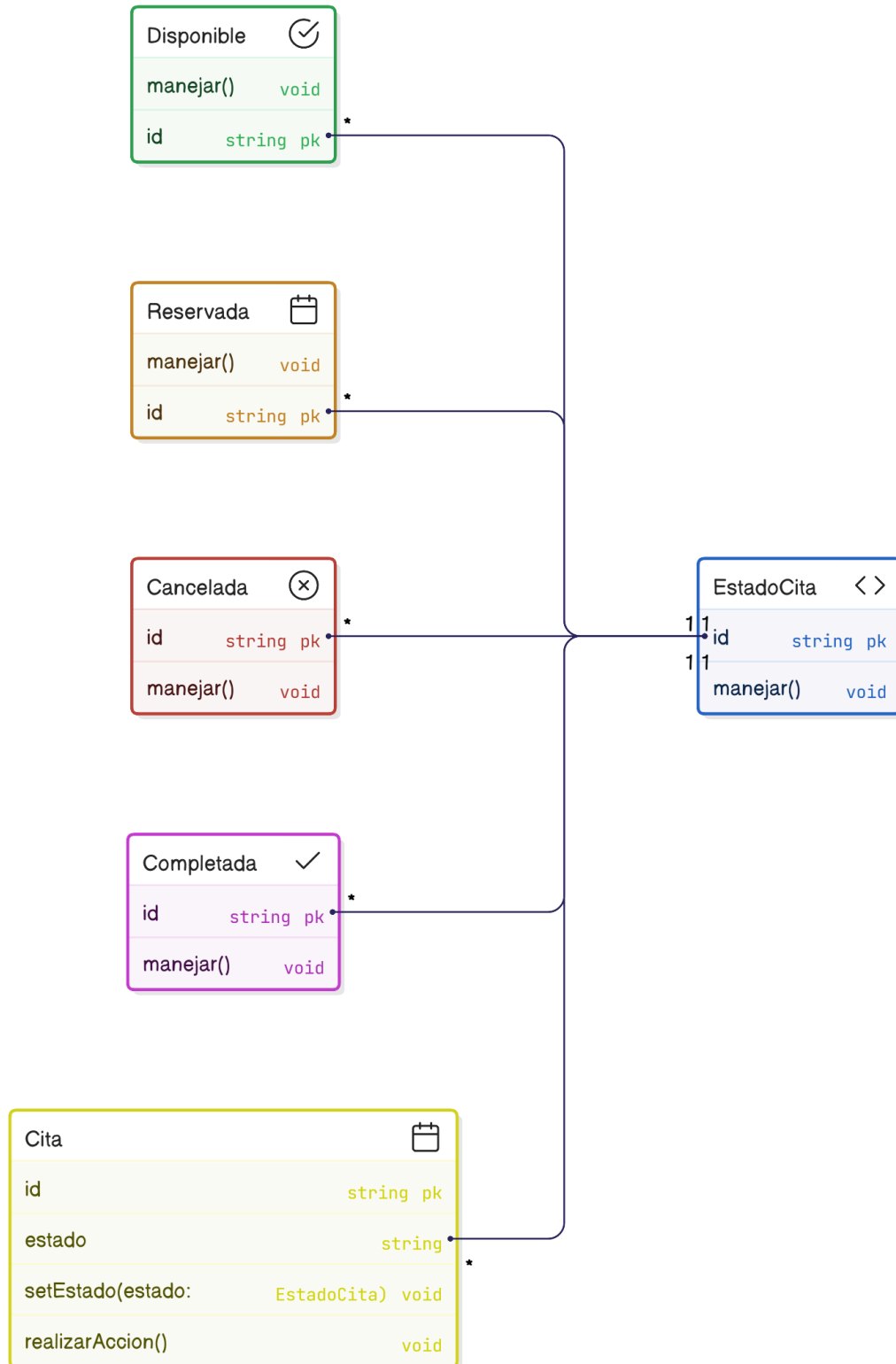
Para mejorar la estructura y escalabilidad del sistema, se implementó el **patrón de diseño State**. Este patrón permite encapsular los comportamientos asociados a un estado particular dentro de clases separadas, haciendo que el objeto que cambia de estado (en este caso, una `Cita`) **delegue el comportamiento a su estado actual**. Así, cada estado define su propio comportamiento y facilita la transición entre estados de forma limpia y controlada.

3. Participantes

- **State (EstadoCita):** Interfaz común para todos los estados. Declara métodos que serán implementados por los estados concretos.
 - **Concrete States (Disponible, Reservada, Cancelada, etc.):** Implementan el comportamiento específico para cada estado.
 - **Contexto (Cita):** Contiene una referencia al objeto `EstadoCita` y delega en él las acciones que dependen del estado. Puede cambiar dinámicamente su estado interno.
 - **Cliente:** La clase que crea y manipula las citas, y solicita cambios de estado según las acciones del usuario o el sistema.
-

4. Diagrama UML

Modelo de estados de una cita médica (Patrón State)



5. Código

```
* and open the template in the editor.
*/
package Vista_Doctor;

public class CitaContexto {
    private EstadoCita estado;

    public void setEstado(EstadoCita estado) {
        this.estado = estado;
    }

    public void mostrar(javax.swing.JComboBox<String> horaBox, String hora) {
        estado.mostrarEstado(horaBox, hora);
    }
}
```

```
History
package Vista_Doctor;
public interface EstadoCita {
    void mostrarEstado(javax.swing.JComboBox<String> horaBox, String hora);
}
```

```

*/
package Vista_Doctor;

public class EstadoDisponible implements EstadoCita {
    @Override
    public void mostrarEstado(javax.swing.JComboBox<String> horaBox, String hora) {
        horaBox.addItem("⏰ Disponible - " + hora);
    }
}

```

```

package Vista_Doctor;

public class EstadoReservada implements EstadoCita {
    @Override
    public void mostrarEstado(javax.swing.JComboBox<String> horaBox, String hora) {
        horaBox.addItem("⏰ Reservada - " + hora);
    }
}

```

```

}

private void FechaActionPerformed(java.awt.event.ActionEvent evt) {
    Hora.removeAllItems();
    fecha = Fecha.getSelectedItem().toString();
    Hora.setEnabled(true);

    // Obtener lista de horas disponibles de tu método actual
    List<String> horas = Formato_Citas.obtenerHorasDisponiblesLista(conn, fecha, idMedico);
    List<String> horasReservadas = Formato_Citas.obtenerHorasReservadasLista(conn, fecha, idMedico);

    CitaContexto contexto = new CitaContexto();

    for (String h : horas) {
        if (horasReservadas.contains(h)) {
            contexto.setEstado(new EstadoReservada());
        } else {
            contexto.setEstado(new EstadoDisponible());
        }
        contexto.mostrar(Hora, h);
    }
}

```

6. Conclusión

Gracias al patrón State, se logró una gestión de estados de las citas **más clara, escalable y mantenible**. Se eliminó la necesidad de grandes bloques de condicionales para manejar la lógica específica de cada estado, lo cual facilita futuras modificaciones o la adición de nuevos estados. Este patrón es ideal en sistemas donde los objetos cambian su comportamiento dinámicamente según su estado interno, como en flujos de trabajo, ciclos de vida o procesos por etapas.